

Underwater Fish and Marine Animal Detection with YOLOv8

Emre Yoleri

Technical Report · October 2026

Abstract. Automatic detection of fish and other marine animals in underwater footage is needed for biodiversity monitoring and aquarium and fisheries analytics, but underwater images are degraded by a blue–green colour cast, haze, blur and low light. This report presents a detection system based on the YOLOv8s one-stage detector, fine-tuned from COCO weights on the public Roboflow Aquarium dataset with seven classes (fish, jellyfish, penguin, puffin, shark, starfish and stingray). The training set is doubled with offline, box-preserving augmentations that simulate underwater degradation (colour cast, turbidity, blur and low light), and an OpenCV preprocessing module (red-channel compensation, white balance and CLAHE) is provided for inference. After a short training schedule of 10 epochs on a laptop GPU, the model reached a precision of 0.752, a recall of 0.654, an mAP50 of 0.727 and an mAP50-95 of 0.414 on the held-out test set (63 images, 582 objects), and processed a 1280×720 aquarium video at about 15 frames per second.

Keywords: object detection, YOLOv8, underwater imaging, fish detection, data augmentation, mAP.

1. Introduction

Cameras are increasingly used to monitor marine life in reefs, fish farms and aquariums, and the amount of video they produce is too large for manual annotation [1], [2]. Underwater images are, however, difficult for detectors: water absorbs red light and gives the scene a blue–green cast, suspended particles scatter light and reduce contrast [3], and moving animals and cameras cause motion blur. In addition, animals are often small, partly occluded and densely grouped.

One-stage detectors of the YOLO family [4] predict all boxes in a single network pass and are fast enough for real-time video. The objective of this work is to (i) fine-tune a YOLOv8 detector [5] for seven classes of marine animals, (ii) add underwater-specific data augmentation and preprocessing built with OpenCV and NumPy, (iii) evaluate the detector with standard metrics — precision, recall and mean average precision (mAP) — overall and per class, and (iv) measure its speed on real video.

2. Methodology

2.1. Dataset

The Roboflow Aquarium dataset, obtained from the public Roboflow 100 benchmark [6], contains photographs taken in two aquariums in the United States, annotated with bounding boxes for seven classes. The COCO-format export was converted to YOLO format (normalised centre, width and height), and Roboflow’s empty super-category was removed. The official split was kept: 448 training, 127 validation and 63 test images. Table 1 gives the number of annotated objects per class. The data is strongly imbalanced: fish account for about half of all boxes, while starfish and stingray appear only 11 and 15 times in the test set.

Table 1. Number of images and annotated objects per split. The training split contains each of the 448 original images and one augmented copy.

Split	Images	Fish	Jellyfish	Penguin	Puffin	Shark	Starfish	Stingray
Train (+aug.)	896	1964	385	330	175	259	78	136
Validation	127	459	155	104	74	57	27	33
Test	63	249	154	82	35	36	11	15

2.2. Underwater Augmentation and Preprocessing

To make the detector robust to the variation found under water, one augmented copy of every training image was generated offline (fixed random seed). Each copy is horizontally flipped with probability 0.5 (boxes are mirrored accordingly) and then receives one or two randomly chosen photometric transforms:

a **colour cast** that scales the B, G and R channels by random gains in $[1.0, 1.25]$, $[0.95, 1.2]$ and $[0.6, 0.95]$; **turbidity**, which blends the image with a random haze colour A as $I' = tI + (1 - t)A$ with $t \in [0.6, 0.9]$, a simplified form of the scattering model [3]; Gaussian or horizontal motion **blur** with a 3–7 pixel kernel; and **low light**, a gamma curve with $\gamma \in [1.4, 2.0]$ followed by Gaussian noise. Photometric transforms do not move objects, so the labels stay valid. During training, Ultralytics applies its own online augmentation on top (mosaic [7], HSV jitter, scaling and translation).

For inference, an optional OpenCV preprocessing step is available: red-channel compensation from the green channel [8], Gray-World white balance [9] and CLAHE [10] on the L^* channel of CIELAB (clip limit 2.0, 8×8 tiles). Figure 1 shows an example of each transform.

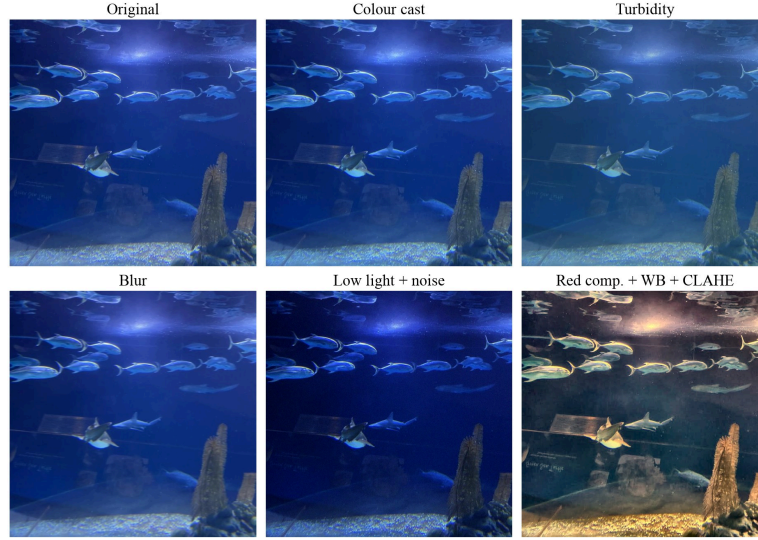


Figure 1. Underwater augmentations used for training (first five panels) and the optional colour and contrast correction used before inference (last panel).

2.3. Model and Training

YOLOv8s (11.1 M parameters, 28.4 GFLOPs) was chosen as a compromise between accuracy and speed. It uses a CSP-based backbone, a feature-pyramid neck and an anchor-free decoupled head; boxes are trained with CIoU and distribution focal loss [11] and classes with binary cross-entropy. The network was initialised with weights pre-trained on COCO [12] and fine-tuned at an input size of 640×640 with a batch size of 16 on an Apple M3 Pro GPU (MPS backend). The optimiser was AdamW [13] with a learning rate of 9.1×10^{-4} (selected automatically by Ultralytics), weight decay 5×10^{-4} and a cosine learning-rate schedule.

Because of limited compute time, training was done in two short stages. Stage 1 ran 4 epochs of a planned 50-epoch schedule, including the 3-epoch learning-rate warm-up. Stage 2 continued from the last stage-1 weights for 6 epochs without warm-up, with the cosine schedule decaying the learning rate to near zero and mosaic augmentation switched off for the last 3 epochs. The weights with the best validation fitness were kept.

2.4. Video Inference and Evaluation

For video, each frame is optionally preprocessed, passed to the detector (confidence 0.25, NMS IoU 0.5), annotated and written as H.264 MP4; the same detector powers a Streamlit application for images, videos and webcam input. Latency was measured from preprocessing to the end of non-maximum suppression; the end-to-end frame rate also includes decoding, drawing and encoding.

Detection quality was measured with the standard metrics [12], [14]. A prediction is a true positive if its IoU with an unmatched ground-truth box of the same class exceeds a threshold. Precision $P = TP / (TP + FP)$ and recall $R = TP / (TP + FN)$ are reported at the confidence that maximises $F_1 = 2PR / (P + R)$. The average precision (AP) is the area under the precision–recall curve; mAP50 is the

mean AP over classes at IoU 0.5, and mAP50-95 averages it over IoU thresholds 0.5:0.05:0.95, which also rewards precise localisation (evaluation used confidence 0.001 and NMS IoU 0.6).

3. Results

3.1. Training

Figure 2 shows the losses and validation mAP per epoch. In stage 1 the validation mAP50 stayed at 0.33–0.42 during the learning-rate warm-up. At the start of stage 2 it dropped to 0.22, because the resumed run started directly at the full learning rate without warm-up, but it then rose in every epoch to 0.693 at epoch 10 while all losses decreased. The curve was still rising at the end, which shows that the model had not converged.

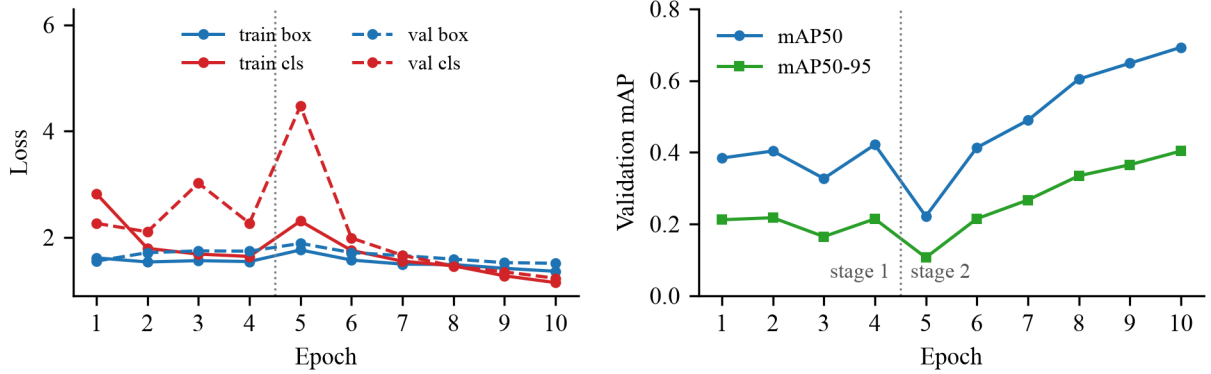


Figure 2. Training and validation losses (left) and validation mAP (right) over the two training stages. The dotted line marks the start of stage 2.

3.2. Detection Accuracy

Table 2 summarises the results. On the test set the detector reached $P = 0.752$, $R = 0.654$ ($F_1 = 0.699$), mAP50 = 0.727 and mAP50-95 = 0.414. The test scores are slightly above the validation scores, so there is no sign of overfitting to the split used for model selection.

Table 2. Overall detection results of YOLOv8s.

Split	Precision	Recall	F1	mAP50	mAP50-95
Validation (127 img., 909 obj.)	0.747	0.646	0.693	0.702	0.403
Test (63 img., 582 obj.)	0.752	0.654	0.699	0.727	0.414

Table 3. Per-class results on the test set.

Class	P	R	mAP50	mAP50-95
Fish	0.790	0.614	0.744	0.432
Jellyfish	0.783	0.799	0.847	0.519
Penguin	0.759	0.622	0.718	0.313
Puffin	0.812	0.371	0.524	0.197
Shark	0.794	0.750	0.815	0.491
Starfish	0.907	0.886	0.922	0.576
Stingray	0.416	0.533	0.520	0.367
All	0.752	0.654	0.727	0.414

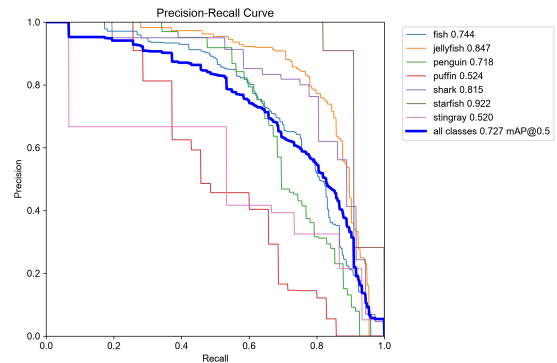


Figure 3. Precision–recall curves on the test set.

Per-class results (Table 3, Figure 3) differ strongly. Starfish (mAP50 = 0.922), jellyfish (0.847) and sharks (0.815) are detected best: they have distinctive shapes and, in the case of jellyfish, high contrast against the blue background. The weakest classes are puffin (0.524) and stingray (0.520). Puffins are small and often stand in dense groups on rocks above the water, which gives a high precision (0.812) but the lowest recall (0.371). Stingrays are flat and seen from many angles, often lying on the sand with little contrast; with only 136 training boxes (including augmented copies) and 15 test boxes, their scores

are also the least reliable. Fish, the largest class, reach a high precision (0.790) but a moderate recall (0.614), mainly because many small and overlapping fish in schools are missed.

3.3. Qualitative Results and Video Speed

Figure 4 compares predictions with the ground truth on four test images and shows two frames of the processed video. Sharks, penguins and jellyfish are localised accurately; the misses are mostly small or partly hidden animals, and one small false jellyfish box appears in the third image. In the puffin image, the swimming bird receives two overlapping boxes, one of them much too large. In the aquarium video, clownfish in an anemone are found with moderate confidence, showing that the model generalises to new scenes, although some fish are missed and the confidence is often below 0.5.

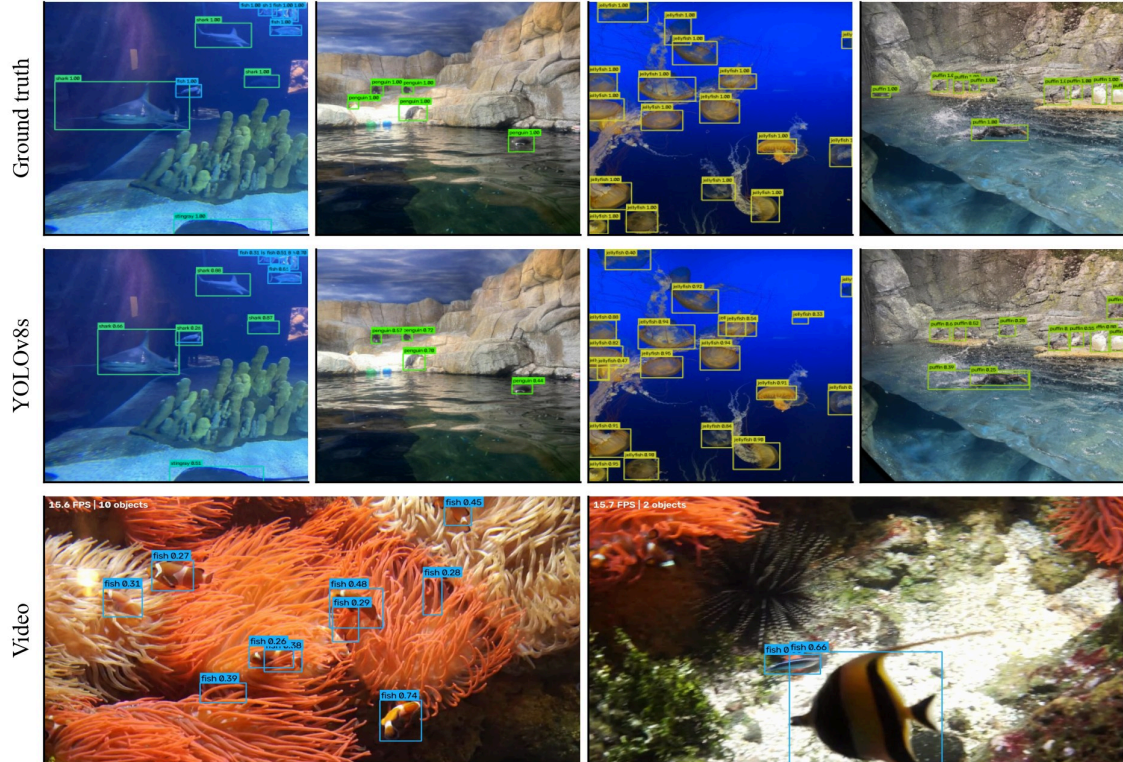


Figure 4. Ground truth (top) and YOLOv8s predictions (middle, confidence ≥ 0.25) on four test images, and two frames of the annotated aquarium video (bottom).

The video has 1226 frames at 1280×720 pixels. The mean latency per frame was 65.9 ms (95th percentile 77.8 ms), i.e. 15.2 frames per second for the model and 14.2 frames per second end to end, on the Apple M3 Pro. On average 3.8 objects were detected per frame; 4577 of the 4617 detections were labelled as fish, and the remaining 40 were starfish, jellyfish, puffin and shark labels; since the clip shows mainly reef fish, corals and anemones, most of these are probably misclassifications. On the 640×640 test images, the network itself needed only 6.9 ms per image (batch 16), so a large part of the video latency is probably overhead from per-frame resizing, a batch size of one and host-GPU transfer.

4. Discussion

The results show that a small, COCO-pretrained one-stage detector can be adapted to underwater scenes with very little training: after only 10 epochs, about 17 minutes on a laptop, it reached an mAP50 of 0.727 on unseen test images and ran at about 15 frames per second on high-definition video. The per-class analysis shows that accuracy depends more on object size, crowding and the number of training examples than on the underwater colour cast itself: large or high-contrast animals (sharks, jellyfish, starfish) are detected well, while small, crowded objects (puffins, schooling fish) mainly suffer from low recall. The

gap between mAP50 and mAP50-95 (0.727 vs. 0.414) also shows that boxes are usually found but are not always tightly localised, especially for flat stingrays and small puffers.

The work has several limitations. First, the model was not trained to convergence: the validation mAP was still rising by about 0.04 per epoch at the end, and the training was split into two stages with a learning-rate jump at the restart. Second, the dataset is small and imbalanced, and the per-class test scores for starfish and stingray rest on only 11 and 15 objects. Third, the effect of the underwater augmentation and of the CLAHE preprocessing was not measured separately, because only one model (trained on raw images with augmentation) was evaluated; the preprocessing is implemented but its benefit is not proven. Finally, all images come from aquariums, whose water is clearer than the open sea.

5. Conclusion

A YOLOv8s-based system for detecting seven classes of marine animals was developed with Python, OpenCV and Ultralytics, including underwater-specific augmentation, optional colour and contrast correction, video inference and a Streamlit application. On the held-out test set of the Roboflow Aquarium dataset it achieved a precision of 0.752, a recall of 0.654, an mAP50 of 0.727 and an mAP50-95 of 0.414, and it processed 1280×720 video at about 15 frames per second on a laptop GPU. Future work includes a full training schedule of 50 or more epochs, an ablation that compares training with and without the underwater augmentation and CLAHE preprocessing, a larger input size or tiled inference for small objects, and the addition of object tracking to count individual animals across video frames.

References

- [1] A. Salman *et al.*, “Automatic Fish Detection in Underwater Videos by a Deep Neural Network-Based Hybrid Motion Learning System,” *ICES Journal of Marine Science*, vol. 77, no. 4, pp. 1295–1307, 2020.
- [2] S. Villon *et al.*, “A Deep Learning Method for Accurate and Fast Identification of Coral Reef Fishes in Underwater Images,” *Ecological Informatics*, vol. 48, pp. 238–244, 2018.
- [3] D. Akkaynak and T. Treibitz, “A Revised Underwater Image Formation Model,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6723–6732.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [6] F. Ciaglia, F. S. Zuppichini, P. Guerrie, M. McQuade, and J. Solawetz, “Roboflow 100: A Rich, Multi-Domain Object Detection Benchmark,” *arXiv preprint arXiv:2211.13523*, 2022.
- [7] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [8] C. O. Ancuti, C. Ancuti, C. De Vleeschouwer, and P. Bekaert, “Color Balance and Fusion for Underwater Image Enhancement,” *IEEE Transactions on Image Processing*, vol. 27, no. 1, pp. 379–393, 2018, doi: 10.1109/TIP.2017.2759252.
- [9] G. Buchsbaum, “A Spatial Processor Model for Object Colour Perception,” *Journal of the Franklin Institute*, vol. 310, no. 1, pp. 1–26, 1980.
- [10] K. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization,” in *Graphics Gems IV*, P. S. Heckbert, Ed., Academic Press, 1994, pp. 474–485.
- [11] X. Li *et al.*, “Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 21002–21012.
- [12] T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [13] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.
- [14] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.